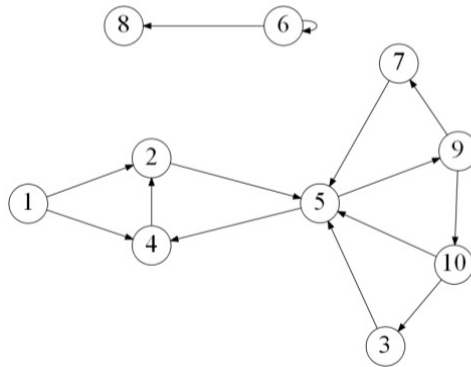




1. Draw how the graph would look if represented by an adjacency matrix. You may assume the indexes are from 1 through 10. Indicate 1 if there is an edge from vertex A $\rightarrow$ vertex B, and 0 otherwise. (10 points)

The graph above represented as an adjacency matrix:

	1	2	3	4	5	6	7	8	9	10
1	0	1	0	1	0	0	0	0	0	0
2	0	0	0	0	1	0	0	0	0	0
3	0	0	0	0	1	0	0	0	0	0
4	0	1	0	0	0	0	0	0	0	0
5	0	0	0	1	0	0	0	0	1	0
6	0	0	0	0	0	1	0	1	0	0
7	0	0	0	0	1	0	0	0	0	0
8	0	0	0	0	0	0	0	0	0	0
9	0	0	0	0	0	0	1	0	0	1
10	0	0	1	0	1	0	0	0	0	0

2. Draw how the graph would look if represented by an adjacency list. You may assume the indexes are from 1 through 10. (10 points)

The graph above represented as a adjacency list :

1	2, 4
2	5
3	5
4	2
5	4, 9
6	6, 8
7	5
8	
9	7, 10
10	3, 5

Total edges
amount to 14 ✓

3. List the order in which the vertices are visited with a breadth-first search. If there are multiple vertices adjacent to a given vertex, visit the adjacent vertex with the lowest value first. (10 points)

To list the order of BFS (breadth-first search):

we start at some vertex and search the graph, outputting the order which each vertex is visited.

Starting at vertex 1 and traverse the graph

In this case, we have an array of size 10 (10 vertices) and each vertex has a counter and we record when each vertex is visited.

We also use a queue (FIFO) to track what vertices were visited, and what we need to visit next.

If a vertex is already visited, we don't update visit or queue.

Start at bfs(1) $\left\{ \begin{array}{l} \text{record visit} \\ \text{Increment count} \\ \text{update queue} \end{array} \right\}$ continue until queue is empty or all vertices visited

Array/vertices

	1	2	3	4	5	6	7	8	9	10
visit track	0	0	0	0	0	0	0	0	0	0
Using Index	1	2	8	3	4	9	6	10	5	7
Count:	0	1	2	3	4	5	6	7	8	9
Queue:		1	2	4	5	9	7	10	3	6

Now, we re-write vertices in order they were visited

1, 2, 4, 5, 9, 7, 10, 3, 6, 8

4. List the order in which the vertices are visited with a depth-first search. If there are multiple vertices adjacent to a given vertex, visit the adjacent vertex with the lowest value first. (10 points)

To list the order of vertices visited using (DFS) depth-first search:

we use an approach similar to BFS, but traverse the graph in a different manner.

we have an array of which vertex we've seen in the order we see; We don't use a queue, we use a stack (LIFO).

This time we traverse all the way and backtrack / pop items off the stack

Array/vertices

	1	2	3	4	5	6	7	8	9	10	
visit track	0	0	0	0	0	0	0	0	0	0	10
Using Index	1	2	8	4	3	9	6	10	5	7	dfs(8)
Count:	0	1	2	3	4	5	6	7	8	9	
Stack:	dfs(1)	dfs(2)	dfs(5)	dfs(4)	dfs(9)	dfs(7)	dfs(10)	dfs(3)	dfs(6)		

Now, we re-write vertices in order they were visited

1, 2, 5, 4, 9, 7, 10, 3, 6, 8

5. a) What is the running time of breadth-first search with an adjacency matrix? (5 points)
b) What is the running time of breadth-first search with an adjacency list? (5 points)
6. a) What is the running time of depth-first search with an adjacency matrix? (5 points)
b) What is the running time of depth-first search with an adjacency list? (5 points)

5a. The running time of BFS with an adjacency matrix would be $\Theta(V^2)$

b. The running time of BFS with an adjacency list would be $\Theta(V + E)$

6a. The running time of DFS with an adjacency matrix would be $\Theta(V^2)$

b. The running time of DFS with an adjacency list would be $\Theta(V + E)$

7. While an adjacency matrix is typically easier to code than an adjacency list, it is not always a better solution. Explain when an adjacency list is a clear winner in the efficiency of your algorithm? (5 points)

Although an adjacency matrix is typically easier to code, an adjacency list is a clear winner in the efficiency of an algorithm. For an adjacency matrix you always have to have a V by V matrix. Ex) graph with 20 vertices and 50 edges.

Sparse graph

20x20 matrix

If the number of vertices is large compared to the edges, there will be a lot of memory lost (lots of empty cells). In our example, only 50 of the slots would have "1" and the rest would be zero. Therefore, an adjacency list will be more efficient. The opposite would be true if edges would be closer to V^2 , since most cells would be utilized.

400 spots/cells

↳ 1 if edge 0 if not

But only 50 edges

8. Explain how one can use a breadth-first to determine if an undirected graph contains a cycle. (10 points)

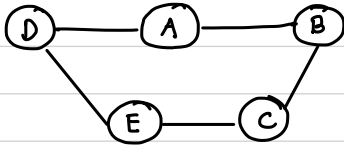
One can use a breadth-first to determine if an undirected graph contains a cycle by starting BFS traversal from any unvisited node in the graph. While traversing, mark each visited node and if a node is encountered that was already visited, this suggests there is a cycle present. Continue BFS until all nodes are visited or a cycle is detected.

Non-parent

- DFS may be suitable for finding cycles in deep, sparse graphs
- BFS may be suitable for dense, wide graphs where cycles are closer to the start

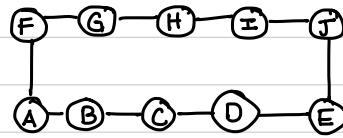
9. On undirected graphs, does either of the two traversals, DFS or BFS, always find a cycle faster than the other? If yes, indicate which of them is better and explain why it is the case; if not, draw two graphs supporting your answer and explain the graphs. (10 points)

No, neither DFS or BFS traversals always find a cycle faster than the other. The speed of which is faster would be dependent on the structure of the graph and location of the cycle within the graph.



- the graph above would be ideal for BFS since you are considering neighbors.

This graph is also not deep, and each node has an equal # of



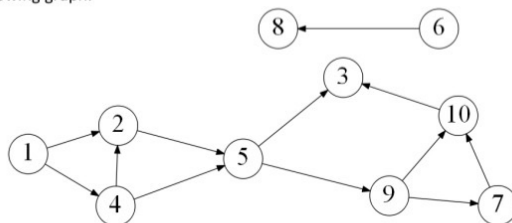
- the graph above would be ideal for DFS since you go through the depth of one path instead of looking at neighbors for a path.

10. Explain why a topological sort is not possible on the graph at the very top of this document. (5 points)

neighbors.

Topological sort is not possible on the graph at the very top because it contains cycles. In the graph at the very top, there is a cycle from node 6 to node 6. The cycle prevents a topological sort because there cannot be a topological sort between two nodes that reference each other. In other words, there is a path that leads back to the same vertex, creating circular dependencies that prevent the establishment of a linear ordering. There are also cycles between nodes 5-9-10, 5-9-7, and 5-4-2.

Consider the following graph:



11. List the order in which the vertices are visited with a topological sort. Break ties by visiting the vertex with the lowest value first. (10 points)

Khan's Algorithm Since there are no edges / indegree directed to node 1 and 6

Vertex	1	2	3	4	5	6	7	8	9	10	
Indegree	0	1	1	1	1	0	1	1	1	1	→ # of edges or indegrees directed to a node in the graph
		0	0	0	0		0	0	0	0	

set $S = \{ \cancel{1}, \cancel{6}, \cancel{1}, \cancel{7}, \cancel{5}, \cancel{1}, \cancel{8}, \cancel{7}, \cancel{10}, \cancel{3} \}$

$L = [1, 4, 2, 5, 6, 8, 9, 7, 10, 3]$

Stack
or
Queue

Therefore, the order of the vertices visited with a topological sort is $[1, 4, 2, 5, 6, 8, 9, 7, 10, 3]$